Can you please find a big list of plant guilds? We want to build a deterministic state machine for crop rotations

Thought for 44s

Here are some **big, ready-made lists of plant guilds / polycultures** you can pull from, plus a way to turn them into a **deterministic crop-rotation state machine**.

Big lists of plant guilds (good "source of truth" compilations)

1) Plants For A Future: "20 Plant Polycultures and Guilds" (temperate-focused)

This is one of the cleanest "big list" pages because it's explicitly a collection of **multiple annual polycultures + multiple tree guilds + multiple perennial herbaceous polycultures**, with a table of contents you can treat like an index. pfaf.org

It includes (as top-level entries) things like:

- **Annual polycultures:** "Simple companion group", "Four Sisters", multiple named annual polycultures pfaf.org
- **Tree guilds:** Beach plum, pawpaw, apple, red alder, mulberry (and more details under each) pfaf.org
- **Perennial herbaceous polycultures:** multiple numbered herbaceous polycultures + an "insect support group of flowers" pfaf.org

2) "Examples of Edible Forest Garden Guilds" (PDF handout)

A compact PDF-style list of guild examples (nitrogen-fixer marked, etc.) that's easy to scrape into a table. driftless.wisc.edu

3) Midwest Permaculture "Plant Guilds eBooklet" (PDF)

A longer booklet with multiple guild examples (often tree-anchored) and narrative context. midwestpermac... +1

4) "Sample Guilds" (Temperate Climate Food Forest Guilds) (PDF)

A short "copyable" set of guild examples, with the classic functional categories (nitrogen fixers, mineral accumulators, beneficial insect plants, etc.). Permaculture A...

5) Three Sisters / Four Sisters style annual guilds (useful for rotation logic)

- Three Sisters (corn + climbing beans + pumpkin/squash/cucurbit) is documented in multiple gardening references and is a canonical **annual polyculture** building block. ABC News +1
- "Four Sisters" appears as a named annual polyculture in the PFAF compilation's contents. pfaf.org

---

A practical way to make this "deterministic" for crop rotations

Instead of treating "guild" as just a plant list, represent each guild as a **state with a small vector of properties** that drive rule-based transitions.

1) Define a guild record (machine-friendly)

For each guild, store:

- **Anchor crop / crop family** (e.g., maize; solanaceae; brassica; orchard-tree)
- **Primary function outputs** (pick a fixed set of axes, e.g.)
  - `N` (nitrogen gain / fixation)
  - `Pest` (beneficial insect support / repellence)
  - `OM` (organic matter / mulch)
  - `Root` (root-depth mixing / compaction breaking)
  - `Weed` (groundcover / suppression)
- **N demand class**: `{heavy_feeder, moderate, light, builder}`
- **Season window**: `{warm, cool, overwinter}`
- **Bed occupancy**: `{full, partial, understory}` (important if you mix annual beds + perennials)

The key is: **your state machine transitions on these properties**, not on botanical poetry.

2) Deterministic transition rules (example)

You can do strict rules like:

- If `N_demand == heavy_feeder` → next must be a **builder** guild (legume/cover focus)
- If `Pest pressure high` or you want prophylaxis → next must include **insectary** component (the PFAF page even has an "insect support group of flowers") pfaf.org
- If `Root compaction high` → next must include **deep taproot / accumulator** role (many guild guides explicitly call out "dynamic accumulators" like comfrey/chicory/dandelion as a role) Permaculture G... +1
- If state is an **annual polyculture** like Three Sisters → next forbids another high-N-demand guild unless an intervening builder state occurs ABC News +1

That gives you determinism: **given soil flags + last state, the next state is uniquely determined**.

3) Where the guild lists plug in

Use the big compilations as your **catalogue of candidate states**, then normalize each into your record format:

- Start by importing the PFAF "20 plant polycultures and guilds" as your baseline index of named guild states. pfaf.org
- Add PDF guild handouts/booklets as extra states, then tag them with your standardized properties. driftless.wisc.edu +2

If you want a quick "starter state set" (rotation-oriented)

If your goal is crop rotation (not just food-forest design), a minimal deterministic set is usually:

1. **Heavy feeder annual guild** (e.g., Three Sisters / similar warm-season polyculture) ABC News +1

2. **Builder/cover guild** (legume + biomass + soil cover) (roles commonly listed as nitrogen fixers / suppressors / mulch) Permaculture G... +2
3. **Root + pest-break guild** (taproot/accumulator + insectary strip) Permaculture G... +1
4. **Cool-season moderate feeder guild** (brassica/allium/greens mixes—depends on your climate/calendar)

...and then you expand from there with the larger catalogues.

If you tell me (a) your climate band (temperate / subtropical), (b) whether this is **annual beds only** or mixed with perennials, and (c) the rotation period you want (e.g., 4-season loop vs 3-year loop), I can turn the above into an actual **deterministic transition table** (states + guards + next-state function) using these guild sources as the initial library.

Sources

Define the minimal state machine

Here's a **minimal deterministic state machine** for crop rotations that's *actually usable*: **4 states**, each a "guild archetype," with **hard transition rules** so the next state is uniquely determined from current state + a tiny set of flags.

State space

We'll model each bed/plot as being in exactly one state each season.

States (4)

S0 — Build (B)
Soil-building guild: N-fixers + biomass + cover (e.g., vetch/pea/clover + grass/rye/oats + groundcover).

S1 — Heavy Feed (H)
High-demand cash-crop guild (e.g., corn/solanaceae/cucurbits, "big fruiting" crops).

S2 — Pest/Break (P)
Break-cycle guild: botanically "different" from last cash crop + insectary/repellent component (e.g., alliums + herbs/flowers, or brassica mix with insectary).

S3 — Root/Mineral (R)
Deep-root / mineral accumulator + structure repair (e.g., daikon/chicory/comfrey-style role; in annual systems: daikon + phacelia/buckwheat style).

That's the minimal set that covers the core constraints: **fertility, extraction, pathogen/pest breaks, and structure**.

Observations (the only inputs you need)

Per bed, per season, compute 3 flags:

1. **N** (nitrogen status): `low | ok`
2. **C** (compaction/structure): `compacted | ok`
3. **D** (disease/pest pressure): `high | ok`

And one bookkeeping value:

4. **last_cash_family** (optional but very useful): the last major crop family grown in a cash-crop state (H or P when it's a crop).

If you truly want minimal-minimal, you can drop `last_cash_family` and just rotate by state; but pest-break quality is much better with it.

Deterministic transition function

At the end of each season, you move to the next state by **priority rules** (first match wins). This guarantees determinism.

Priority rules (hard)

Rule 1 — If compaction is bad → go to Root/Mineral

- If `C = compacted` then **next = R**.

Rule 2 — Else if nitrogen is low → go to Build

- If `N = low` then **next = B**.

Rule 3 — Else if disease/pest pressure is high → go to Pest/Break

- If `D = high` then **next = P**.

Rule 4 — Else (everything ok) → go to Heavy Feed

- If `C = ok` and `N = ok` and `D = ok` then **next = H**.

That's the whole transition logic.

What each state "emits" (how it updates flags)

To keep the machine closed, you need simple update expectations. You can implement these as deterministic updates or as "targets" you verify with measurements.

State effects (simple defaults)

B (Build):

- Sets `N := ok`
- Tends to set `D := ok` (more diversity/beneficials)
- `C := ok` (some improvement, but not guaranteed if severely compacted)

R (Root/Mineral):

- Sets `C := ok` (primary purpose)
- Leaves `N` unchanged or slightly improves (depending on mix)
- `D` often improves (rotation effect)

P (Pest/Break):

- Sets `D := ok`
- `N` modest draw (often `N := ok` if it was ok; can become low if you pick moderate feeders)

H (Heavy Feed):

- Sets `N := low` (assume extraction)
- May set `D := high` if repeated family pressure (see below)
- `C` can drift toward compacted depending on traffic/soil

Optional "family-repeat" disease trigger (still deterministic)

If you track `last_cash_family`, you can deterministically set disease pressure:

After **H**, set:

- `D := high` **iff** the next chosen cash crop is same family as `last_cash_family`
- otherwise `D := ok`

This is where your guild library plugs in: each H/P guild declares a dominant family.

Minimal output: the state transition table

Given the priority rules, the transition table is:

- From **any state**, if `C=compacted` → **R**
- Else if `N=low` → **B**
- Else if `D=high` → **P**
- Else → **H**

So the "from-state" doesn't matter; only the flags do. That's fine: it's still a state machine (Moore-style with observed inputs), and it's minimal for determinism.

How to turn this into crop rotations immediately

Assign 1–3 concrete guilds per state

- **B:** legume + cereal cover + living mulch
- **R:** daikon + phacelia/buckwheat (annual) / comfrey role (perennial edges)
- **P:** allium/herb/insectary strip, or brassica break mix (choose opposite of last family)
- **H:** your cash crop guilds (tomato guild, corn/bean/squash, etc.)

Rotation emerges automatically

You don't "plan a rotation"; you run the machine per bed each season.

If you want the absolute minimal (no flags)

If you want **purely cyclic** and deterministic with *zero* soil flags:

B → H → P → R → B → …

That's a 4-cycle. It's simpler, but it will be wrong sometimes (e.g., you'll heavy-feed when N is low).

If you tell me whether you're doing **annual beds only** (market garden style) or a **mixed perennial system**, I'll give you a concrete "one guild per state" starter set that fits, but the minimal machine above is already complete and deterministic.

We want something that computes most profitable rotations that maintain soil

Got it. What you want is still a **state machine**, but with an **optimizer** sitting on top of it:

- the *state* summarizes soil condition + rotation memory,
- *actions* are "choose next crop/guild,"
- transitions update soil deterministically,
- and we pick the action sequence that **maximizes profit** subject to **soil staying above thresholds**.

Below is a minimal, clean formulation that you can implement as **dynamic programming** (deterministic) or **integer programming** (exact, scalable).

---

1) Minimal state for "maintain soil" + rotation constraints

Use a small discrete state so it stays tractable.

Soil state (3 scalars, discretized)

Let each be an integer bucket, e.g. 0..4.

- **N** = plant-available nitrogen / fertility bucket
- **C** = carbon / organic matter bucket
- **S** = structure (compaction/aggregation) bucket

So a soil state is `(N, C, S)`.

Rotation memory (1 scalar)

To enforce "don't repeat families," you need memory:

- **F** = last crop family (or last "pressure class"), from a small set like `{none, solanaceae, brassica, cucurbit, allium, legume, cereal, …}`

So full state is:

$$x_t = (N_t, C_t, S_t, F_t)$$

This is basically the minimal Markov state that still captures "soil" and "rotation".

---

2) Actions = candidate crops or guilds

Each action $a \in A$ is a crop/guild option with attributes:

- `profit[a]` (expected gross margin per bed per season)
- `deltaN[a], deltaC[a], deltaS[a]` (soil updates)
- `family[a]`
- optional: `duration[a]` if multi-season, `labor[a]`, `risk[a]`, etc.

Deterministic soil transition:

$$(N', C', S') = \mathrm{clip}\big((N, C, S) + (\Delta N[a], \Delta C[a], \Delta S[a])\big)$$

and

$$F' = family[a]$$

Where `clip` keeps values in 0..4.

---

3) "Maintain soil" as hard constraints

Pick minimum acceptable soil buckets:

- $N \geq N_{\min}$
- $C \geq C_{\min}$
- $S \geq S_{\min}$

Constraint must hold for all times t (or at least terminally / on average).

Hard feasibility rule:

- an action is **allowed** in state x only if the resulting x' satisfies minima.

Also enforce rotation rules (examples):

- forbid repeating same family: `family[a] != F`
- or add "cooldown counters" if you want 2–3 season breaks (still small state).

---

4) Objective: maximize profit over a horizon

For a horizon of T seasons, maximize:

$$\max_{a_0,\ldots,a_{T-1}} \sum_{t=0}^{T-1} profit[a_t]$$

subject to feasibility constraints.

This computes your "most profitable rotation that maintains soil."

---

5) Deterministic optimizer: Dynamic Programming (fast to implement)

Because transitions are deterministic, this is straightforward.

Define value function:

$$V_t(x) = \max_{a \in A(x)} \big(profit[a] + V_{t+1}(f(x,a))\big)$$

with $V_T(x) = 0$.

This returns:

- optimal profit
- and the actual rotation (argmax action sequence)

Complexity is basically:
`O(T * |States| * |Actions|)`.

With buckets 0..4 and ~8 families:

- `|States| = 5^3 * 9 = 1125`
  That's tiny. You can run lots of scenarios.

Add "soil value" if you want to avoid edge-hugging

Hard constraints can lead to "just barely maintain soil."
Add a terminal bonus:

$$V_T(x) = \lambda_N N + \lambda_C C + \lambda_S S$$

This makes it prefer healthier soil while still profit-maximizing.

6) Exact optimization alternative: ILP / MIP (good for many beds + coupling)

If you have many beds and constraints like “don’t plant more than X beds of tomatoes,” DP per-bed won’t capture coupling. Use a MIP:

Binary decision:

- $y_{t,a} \in \{0,1\}$: choose action a at time t (per bed or per block)

Constraints:

- $\sum_a y_{t,a} = 1$
- soil dynamics as linear constraints (with precomputed deltas)
- minima constraints
- rotation constraints
- acreage/market constraints across beds

Objective:

- maximize $\sum_{t,a} profit[a] y_{t,a}$

MIP gives global optimum with cross-bed constraints.

---

7) What you need to define (data schema)

For each guild/crop action a:

1. **Profit**: expected gross margin (or net, your choice)
2. **Soil deltas** (per season):
   - heavy feeders: $\Delta N < 0$, often $\Delta C \leq 0$
   - covers/legumes: $\Delta N > 0$, $\Delta C > 0$
   - deep roots: $\Delta S > 0$ (structure improvement)
3. **Family** (or pest-pressure class)

You don’t need perfect soil science initially. Start with coarse deltas like `-2..+2` and calibrate from observation.

---

8) Minimal “starter action set” that already works

You can get good rotations with just 6–10 actions:

Cash crops (profit+)

- H1: Solanaceae cash (tomato/pepper) ΔN=-2 ΔC=-1 ΔS=-1 family=solanaceae
- H2: Cucurbit cash (squash) ΔN=-2 ΔC=-1 ΔS=-1 family=cucurbit
- H3: Brassica cash (broccoli) ΔN=-1 ΔC=-1 ΔS=0 family=brassica
- H4: Allium cash (onion/garlic) ΔN=-1 ΔC=-1 ΔS=0 family=allium

Soil builders (profit low or neutral, but enable future profit)

- B1: Legume-heavy cover ΔN=+2 ΔC=+1 ΔS=0 family=legume
- B2: Biomass cover (rye/oats) ΔN=0 ΔC=+2 ΔS=+1 family=cereal
- R1: Bio-drill mix (daikon etc.) ΔN=0 ΔC=+1 ΔS=+2 family=brassica (or “rootmix”)

DP will naturally choose sequences like:
cash → cover → cash → structure repair → cash … depending on soil buckets.

---

9) The “state machine” view (deterministic)

This is a deterministic automaton:

- **States**: `(N,C,S,F)`
- **Inputs/actions**: crop/guild choice
- **Transition**: add deltas + update family + clip
- **Accepting states**: those with `N,C,S ≥ mins`
- **Controller**: DP/MIP chooses actions to maximize profit while never leaving accepting set.

So it is literally a deterministic state machine with an optimal policy.

If you want, I can write the DP in a way that matches your “DASHI-ish” vibe (explicit state lattice, transition monoid, feasibility as invariant predicate), but the core minimal machine above is the correct computational skeleton.

If you tell me:

- how many seasons in your planning horizon (e.g., 8 seasons = 4 years),
- and your crop list (even rough: tomatoes, brassicas, cucurbits, legumes, grains/cover),
  I’ll plug it into this structure and give you a concrete rotation table template (and the exact DP recurrence tailored to your action set).

yes and pull some figures for prices and soil degradation and try and solve for 1 year with say 10 crop pallette

Thought for 2m 28s

Below is a **minimal deterministic state machine** that can (a) enforce rotation/soil rules and (b) compute the **most profitable 1-year rotation** over a **10-crop palette**, using **real wholesale price figures** (Australia) plus a **simple soil degradation model** tied to **nutrient removal** and **soil carbon / disturbance**.

I’m going to assume you’re operating in an **Australian context** (you’re in AU; Melbourne wholesale markets are a common reference).

---

1) A “big list” of plant guild templates (building blocks)

These are **templates** you can parameterize (swap species/cultivars by climate/market), not one-off “named guilds”:

Tree / perennial guilds

- **Apple/pear orchard guild** (tree + N-fixer shrub + dynamic accumulator + insectary strip + groundcover)
- **Citrus guild**
- **Stone fruit guild**
- **Nut tree guild**

- **Vine guild** (grape/kiwi + living mulch + insectary + sacrificial trap plant)
- **Windbreak guild** (multi-strata: tall nurse + mid canopy + shrub + grass strip)

Annual / market-garden guilds

- **Three Sisters** (maize + climbing bean + squash groundcover)
- **Heavy feeder + undersown clover** (cash crop + living mulch)
- **Brassica + allium companion band** (pest-disruption striping)
- **Salad mix guild** (leafy + fast herbs + edible flowers + trap plant edges)
- **Cucurbit sprawl guild** (trellis cucumber + squash groundcover + insectary)
- **Solanaceae support guild** (tomato + basil/flower strip + mulch crop + trap crop)

Soil-functional guilds

- **N-building cover guild** (vetch/peas/clover) → adds N + biology
- **Carbon pump guild** (rye/sorghum/millet) → adds biomass/roots
- **Biofumigant guild** (mustard/brassica cover) → disease suppression (context-dependent)
- **Deep miner guild** (comfrey/chicory/daikon radish) → break compaction / scavenge nutrients
- **Mycorrhiza support guild** (avoid brassicas; use grasses/legumes + low disturbance)
- **Pollinator/insectary guild** (alyssum/phacelia/buckwheat strips) → beneficials
- **Trap-barrier guild** (nasturtium, sacrificial borders, barrier herbs)

Pest/disease-management guilds (rotation-safe)

- **Family-break guild** (force non-host sequence: e.g., Brassica → Allium → Cucurbit → Legume)
- **Nematode-suppress guild** (tagetes/marigold bands; highly context-specific)
- **Wet-season resilience guild** (raised beds + cover crop roots + minimal soil exposure)

You'll use these guild "types" as **actions** in the state machine (some are cash crops, some are regenerative/cover actions).

---

2) Minimal deterministic state machine (DSM)

State

For 1 bed/block (you can later scale to multiple beds with coupling constraints), define:

$$s_t = (\text{season}_t,\ \text{soil}_t,\ \text{lastFamily}_t)$$

- `season_t ∈ {Q1,Q2,Q3,Q4}` (4 slots in a year; you can do 2–6 slots instead)
- `soil_t` is a small vector of **bounded discrete buckets**, e.g.
  - `N ∈ {0..5}` (plant-available nitrogen proxy)
  - `C ∈ {0..5}` (soil carbon / organic matter proxy)
  - `S ∈ {0..5}` (soil structure/tilth proxy)
- `lastFamily_t` (for disease/pest rotation): Solanaceae, Brassicaceae, Cucurbitaceae, Legume, Allium, etc.

This is deterministic: **given** state + chosen crop action → next state is fixed.

Actions

`a_t ∈ palette` (10 crops/guild-actions; example palette below).

Each action has:

- `family(a)`
- `Δsoil(a) = (ΔN,ΔC,ΔS)`
- `profit(a, season)` (gross margin estimate; can be season-dependent)

Transition

Deterministic update:

1. Rotation rule check (hard constraints), e.g.:
   - no same-family back-to-back
2. Soil update:
   - `soil_{t+1} = clamp(soil_t + Δsoil(a_t), 0..5)`
3. Advance season:
   - `season_{t+1} = next(season_t)`
4. `lastFamily_{t+1} = family(a_t)`

Objective (1 year)

Maximize total profit with soil constraints:

- **Hard constraint** (simple + robust): `soil_t ≥ soil_min` for all t
- Or **soft penalty**: subtract big penalty if soil drops below minimum.

This becomes a small **dynamic program**:

$$V_t(s) = \max_{a \in A(s)} \left[r(s,a) + V_{t+1}(T(s,a)) \right]$$

with $t = 1..4$ for 1 year.

---

3) Concrete 10-crop palette + price figures (wholesale AU)

Here's a workable **10-action palette** using **public wholesale price figures** from Freshlogic articles (these are *category averages by quarter*, good enough to seed the model):

From **Sept 2025 vs Sept 2024 quarter** (wholesale): tomatoes $3.98/kg, potatoes $1.96/kg, pumpkin $1.13/kg, cabbage $1.05/kg, cucumbers $3.23/kg, leeks $3.59/kg, zucchini $1.73/kg. Freshlogic
From **March 2025 quarter**: cucumbers $2.43/kg, beans $7.16/kg, potatoes $1.89/kg, celery $1.39/kg, cabbage $1.22/kg, broccoli $3.18/kg, sweetcorn $3.23/kg. Freshlogic

We'll pick 10 actions:

1. Tomato (Solanaceae) — **$3.98/kg** Freshlogic
2. Zucchini (Cucurbit) — **$1.73/kg** Freshlogic
3. Cucumber (Cucurbit) — **$3.23/kg** (Sep qtr) Freshlogic
4. Broccoli (Brassica) — **$3.18/kg** (Mar qtr) Freshlogic
5. Cabbage (Brassica) — **$1.05–$1.22/kg** Freshlogic +1
6. Potato (Solanum-ish rotation-wise; treat as Solanaceae pressure) — **$1.89–$1.96/kg** Freshlogic +1
7. Pumpkin (Cucurbit) — **$1.13/kg** Freshlogic
8. Beans (Legume) — **$7.16/kg** Freshlogic
9. Leek (Allium) — **$3.59/kg** Freshlogic
10. Cover crop guild: **Ryegrass + vetch** (Regenerative action; revenue ~$0, but soil++)

- Cover crops/OM inputs are highlighted as key to maintaining soil carbon, and intensive veg systems tend to lose carbon via disturbance; an example given is ryegrass cover crop adding **>4 t/ha organic matter** and (with compost) stopping soil carbon decline. Soil Wealth ICP
- Cover crops broadly improve soil properties and can increase soil organic C/N. ScienceDirect

---

4) Soil degradation model seeded from nutrient removal + disturbance

To keep it minimal but not hand-wavy:

Nutrient removal "physics" (data-backed)

The TN88 chart gives **kg nutrient per tonne** crop uptake / edible removal, e.g.

- Tomato (crop uptake) **N 2.8 kg/t**, (fruit) **N 1.5 kg/t** RLF AgTech
- Lettuce (crop uptake) **N 3.0 kg/t** RLF AgTech
- Onion (crop uptake) **N 3.4 kg/t**, bulbs **N 1.54 kg/t** RLF AgTech
- Potato (crop uptake) **N 5.3 kg/t**, tuber removal **N 3.3 kg/t** RLF AgTech
- Pumpkin (fruit removal) **N 1.4 kg/t** RLF AgTech
- Broccoli (edible portion) **N 3.9 kg/t** RLF AgTech

You don't need perfect agronomy to start: use these as **relative drawdown weights**.

Carbon/structure degradation (system-level)

In intensive veg, Soilwealth notes:

- maintaining/increasing soil carbon is hard due to high disturbance and limited residue return Soil Wealth ICP
- harvest/tillage disturbance promotes carbon loss Soil Wealth ICP +1
  and cover crops can mitigate (example ryegrass cover crop) Soil Wealth ICP

Minimal bucket deltas (example)

Let soil buckets start at `(N,C,S)=(4,4,4)` and minimum allowed is `(2,2,2)`.

A simple deterministic mapping (you'll calibrate later):

- Tomato: `Δ = (-2,-1,-1)` (cash crop, high input/disturbance)
- Potato: `Δ = (-2,-1,-2)` (high N draw + harvest disturbance)
- Broccoli/Cabbage: `Δ = (-1,-1, 0)` (moderate draw)
- Cucurbits (cucumber/zucchini/pumpkin): `Δ = (-1,-1, 0)`
- Leek: `Δ = (-1,-1, 0)`
- Beans (legume cash): `Δ = ( 0,-1, 0)` (still exports nutrients; leave N neutral in the minimal model)
- Cover crop (rye+vetch): `Δ = (+2,+2,+1)` (soil rebuilding action)

This is enough structure for the optimizer to "discover" that **pure cash-cropping crashes soil**, so it must include restorative actions.

---

5) Solve 1 year (4 slots) for "most profitable rotation that maintains soil"

Hard constraints

- Soil must stay ≥ (2,2,2) at all times.

- No same-family back-to-back (you can also add “no Solanaceae twice in 3 slots”, etc.)

What the DP will tend to choose (intuitively)

- Beans have the highest $/kg in the public data we pulled ($7.16/kg). Freshlogic
- Tomatoes are strong ($3.98/kg). Freshlogic
- Cucumbers can be strong too ($3.23/kg Sep quarter; $2.43/kg Mar quarter). Freshlogic +1
- But soil constraints force at least one restorative slot (cover crop/OM) in many starting conditions.

Example optimal-feeling 4-slot rotation (with the minimal soil deltas)

Starting soil `(4,4,4)`:

1. **Q1: Tomato** (profitful) → soil `(2,3,3)`
2. **Q2: Beans** (profitful, different family) → soil `(2,2,3)`
3. **Q3: Cover crop (rye+vetch)** (restore) → soil `(4,4,4)`
4. **Q4: Cucumber** (profitful, different family than legume/cover) → soil `(3,3,4)`

This stays above the minima the whole time and “buys” you the ability to run tomato again next year without spiraling downward, consistent with the soil-carbon guidance that cover crops/organic inputs + reduced loss are central in intensive veg systems. Soil Wealth ICP +1

---

6) What you should do next (still minimal, but real)

1. **Decide your time-step**: 4 seasonal slots is fine; market gardens often need 6–8 slots/year (quick crops).
2. Replace “profit per slot” with:
   - `profit = (price $/kg) * (expected yield kg/bed) - variable costs`
3. Replace soil buckets with 3–6 tracked variables:
   - N, C, Structure, DiseasePressure (by family), Salinity/EC if irrigated, Water cost (ABARES weekly update even tracks water allocation prices, which matters in AU). agriculture.gov.au
4. Calibrate `Δsoil` using:
   - nutrient removal weights (TN88) RLF AgTech +1
   - your local soil test trends (to tune C/S loss per cultivation intensity)

If you want, I can also give you a **compact JSON schema** for `state`, `action`, `transition`, and the DP solver interface (so you can drop it straight into code), but the definition above is the minimal “state machine” that still solves the problem you stated.

Sources

Can we model KNF/inputs required as additional input/balance eg if tomatoes need more N but it's easy to grow KNF inputs then that becomes more economical (provided we have space and water etc)

Yes — and it fits *very naturally* into the same deterministic state machine / optimizer. You just promote "fertility" from an implicit soil delta into an explicit **balance sheet** with **purchasable / producible inputs** (including KNF).

There are two clean ways to do it, depending on how "real" you want it to be.

Option A: Treat KNF as a second action layer (inputs as actions)

State (per bed + farm resources)

Per bed you still have soil buckets, but the farm has shared resource stocks:

- Bed state: `(N, C, S, lastFamily)`
- Farm resource state:
  - `KNF_N` = available N-equivalent input units (e.g., "kg N-equiv" or "fertility credits")
  - `Water` (ML or kL)
  - `Labor` (hours)
  - `Space` (m^2 for fermenting / composting / JLF barrels)
  - optional: `Carbon_inputs` (mulch/compost), `Minerals` (Ca/P/K), etc.

So global state is:

$$x_t = \big(\text{bed states}\big) \; + \; (KNF_N_t, Water_t, Labor_t, Space_t)$$

Actions split into

1. **Crop choice** per bed: tomato, broccoli, beans, cover crop…
2. **Input production & application** choices: make/apply KNF inputs, compost, mulch, etc.

Example:

- `make_JLF` : converts (space + labor + water + biomass) → increases `KNF_N`
- `apply_KNF_N_to_bed(i)` : consumes `KNF_N` → increases bed `N` (or offsets crop drawdown)

This stays deterministic because every conversion is fixed.

Feasibility constraints

- Can't apply more `KNF_N` than you have.
- Can't exceed water/labor budgets.
- Space constraints for fermentation/inputs.

Profit function

Profit now includes crop margin **minus**:

- labor cost (or opportunity cost)
- materials cost
- water cost
- (optional) capex amortization for tanks, pumps, etc.

This will automatically discover the trade:

> "Tomatoes draw lots of N, but if KNF_N is cheap for us (because labor/space/water are available), tomatoes become optimal."

Option B: Make nutrients explicit as "demands" and "supplies" (balance model)

This is the most interpretable.

For each crop c

Define per-season:

- `Revenue[c]`
- `N_demand[c]`, `K_demand[c]`, `P_demand[c]` (units)
- `C_impact[c]` and `S_impact[c]` (structure/carbon impacts)
- `Water_demand[c]`
- `Labor_demand[c]`
- `Family[c]`

For each input method m (KNF, compost, purchased fert, legume cover, etc.)

Define:

- `Cost[m]`
- `N_supply[m]`, `K_supply[m]`, `P_supply[m]`
- `Water[m]`, `Labor[m]`, `Space[m]`
- Optional yields side effects (e.g., compost boosts C too)

Then per season you choose:

- crop assignments
- input quantities

Subject to:

- Nutrient balance: supply ≥ demand (or demand is met via soil bucket drawdown)
- Soil health: keep `(C,S)` above minima, optionally penalize low states
- Resource caps: water/labor/space

This can be solved exactly with a MIP (especially if you have many beds) or by DP if single bed.

Minimal "tomato + KNF" example (how it changes optimality)

Assume a single bed, 4 seasons. Soil bucket `N` ranges 0..5.

- Tomato action: profit high, `N_demand = 2` units (or `ΔN = -2`)
- If you *don't* supply N, state drops and constraints force cover crops / lower-profit crops.

Add KNF production:

- `make_KNF_N`: costs 1 labor + 1 water + 1 space → produces `+3 KNF_N`
- `apply_KNF_N`: convert `KNF_N` into `+N` on bed (or cancels tomato drawdown)

Now the optimizer compares:

Path 1 (no KNF): Tomato → soil N crashes → forced builder slot → lower annual profit.

Path 2 (KNF enabled): Make/apply KNF_N → keep N above threshold → can run Tomato + other high-value crops more often → higher annual profit **if** labor/water/space constraints allow and the KNF marginal cost is low enough.

That's exactly your statement, formalized.

The one modeling choice you need to pick

Do you want soil nutrients as…

1. **Buckets** (simple, robust, calibrate later)
   - Tomato consumes 2 “N credits”; KNF adds 2 “N credits”.
2. **Physical units** (kg/ha NPK)
   - More realistic, but you’ll need decent yield assumptions and conversion assumptions.

For a first pass, buckets are perfect and still give correct qualitative behavior.

What I’d implement as the “minimal” KNF extension (recommended)

State

- Bed: `(N, C, S, lastFamily)`
- Global: `(KNF_N, Water, Labor, Space)`

Actions

Per season choose one:

- Crop from palette (10 crops)
- Optional: one KNF production action (or allow multiple with integer units)

Transition (deterministic)

- Crop updates soil: `N -= N_demand[c]`, `C += ΔC[c]`, `S += ΔS[c]`
- Input actions update stocks: `KNF_N += prod`, `Water -=`, `Labor -=`, `Space -=`
- Application updates: `N += applied`, `KNF_N -= applied`

Objective

Maximize:

$$\sum(\text{crop margin}) - (\text{labor cost}) - (\text{water cost}) - (\text{input costs})$$

subject to soil minima and resource budgets.

This will *automatically* pick “tomatoes + KNF” when it’s rational.

If you give me even rough numbers for:

- water limit per season,
- labor hours per season,
- whether you can dedicate e.g. 10–20 m^2 equivalent “space” for barrels/inputs,
- and your 10-crop palette,
  I can write the exact state/action tables (bucket deltas + resource deltas) so the 1-year solver produces an explicit best rotation + input plan.

all_code.txt
Document

Could we integrate livestock?

Thought for 37s ›

Yes — livestock drops in cleanly if you treat them as **mobile nutrient + biomass processors** with explicit **resource balances** (feed/forage → product + manure + soil impact), and you make grazing/stocking an **action** in the same deterministic transition system. all_code

Minimal integration pattern

Extend the state (add 4–6 livestock-relevant scalars)

Keep your bed/field soil state `(N, C, S, lastFamily)` and add:

- `B` = available **forage/biomass** on the paddock (0..5 bucket)
- `M` = **manure/N-credits on-site** (0..5) (or split N/P/K if you want)
- `P` = **parasite/pathogen pressure** (0..5) (important for ruminants)
- `T` = **traffic/poaching risk** (0..5) (wet soil compaction risk)
- Optional shared-farm resources: `Water`, `Labor`, `Fence/Move capacity`, `Bedding`

That's enough to capture "livestock helps soil but can wreck structure if mis-timed".

Add livestock actions (as "rotations" too)

Each season/slot you choose either a crop action or a livestock action (or allow both if you model sub-steps).

Core livestock actions:

- **Graze cover crop (ruminants):**
  Converts `B → manure + cash`, but may reduce `S` if `T` high.
- **Poultry tractor pass (chickens/ducks):**
  Adds `N` (manure), reduces `P` (insect/larvae pressure), but can reduce ground cover (`C`) if overdone.
- **Cut-and-carry to compost / deep litter:**
  Converts `B + Labor + Bedding → compost credits`, later applied as `C/N` gains.
- **Rest / recovery:**
  Increases `B` (regrowth), decreases `P` (break cycle), reduces `T`.

Deterministic transition rules (example)

Let action `a` have fixed deltas:

- Soil: `(ΔN[a], ΔC[a], ΔS[a])`
- Biomass/manure: `(ΔB[a], ΔM[a])`
- Health/traffic: `(ΔP[a], ΔT[a])`
- Profit: `profit[a]` (meat/eggs/milk) minus costs (labor, water, supplements, bedding, fencing moves)

Then:

$$x' = \mathrm{clamp}(x + \Delta(a))$$

Plus **guardrails** (hard constraints), e.g.:

- If `T` high (wet/poaching risk), **forbid grazing** (or force very low stocking action).
- If `P` high, forbid returning same species class too soon (or require "rest" slot).
- Maintain soil minima: `N,C,S ≥ thresholds`.

Why this makes tomatoes more profitable (your KNF idea + livestock)

Livestock gives you additional "input factories":

- **Forage → manure → N/C credits**, reducing purchased fertility or KNF labor.
- **Grazed covers** let you run restorative phases that *also* generate revenue.

So the optimizer can discover rotations like:

1. High-value crop (tomato)
2. Cover crop
3. Graze cover crop (revenue + manure)
4. Medium feeder (brassica/allium)
   …while keeping `(N,C,S)` above thresholds.

The smallest "works in practice" livestock module

If you want the *minimal* add-on that still behaves correctly:

- Add **one** livestock class first (poultry *or* sheep).
- Track only: `B (biomass)`, `M (manure credits)`, `T (traffic risk)`.
- Two actions: `graze` and `rest`.
- One guard: "if `T` high, no graze."

That alone will capture the major economic/soil trade.

If you tell me what livestock you mean (poultry vs sheep/cattle vs pigs) and whether you're doing **cell grazing** or **tractor runs**, I'll give you a concrete 10-action palette that includes crops + livestock + KNF/inputs, with deterministic deltas and constraints suitable for a 1-year solver.

Sources

Can you write our full formalism in complete please?

Absolutely. Here's the **complete formalism** for a **deterministic state machine** that chooses **most-profitable 1-year rotations** while **maintaining soil**, and that can include **KNF / on-farm inputs** and **livestock** as first-class parts of the system.

I'll write it as a spec: **objects → state → actions → transition → constraints → objective → solver → data schema**. You can implement it as DP (single bed) or MIP (many beds + shared resources).

---

0) Sets and indices

- Time steps (within 1 year):

$$t \in \{1, \dots, T\}$$

  Choose T (e.g. 4 seasonal slots, or 6–8 crop windows).
- Beds/fields (optional, for multi-block planning):

$$b \in \{1, \dots, B\}$$

- Crop/guild actions (your palette):

$$c \in \mathcal{C}, \quad |\mathcal{C}| = 10 \text{ (example)}$$

- Input actions (KNF, compost, minerals, purchased fert, irrigation events):

$$i \in \mathcal{I}$$

- Livestock management actions (graze, rest, move poultry tractor, cut-and-carry):

$$\ell \in \mathcal{L}$$

We bundle "what you do at time t" into one **composite action**:

$$a_t = (c_t, i_t, \ell_t)$$

Some components can be "none".

---

1) State

1.1 Bed/field state (per bed b)

A minimal, rotation-capable soil + memory state:

$$x_t^{(b)} = \left(N_t^{(b)},\, C_t^{(b)},\, S_t^{(b)},\, F_t^{(b)},\, D_t^{(b)}\right)$$

Where (use either buckets or physical units):

- N: available fertility / N-credit bucket (or kg/ha available N)
- C: soil carbon/OM bucket (or %SOC / tC/ha proxy)
- S: soil structure bucket (compaction/aggregate proxy)
- F: last crop family / pressure class (categorical)
- D: disease/pest pressure bucket (optional but useful)

Minimal-minimal is (N, C, S, F). D can be derived from F with a rule, but explicit D is cleaner.

1.2 Farm-global resource state (shared across beds)

This is where KNF + livestock constraints live:

$$g_t = (W_t,\, L_t,\, A_t,\, K_t^N,\, K_t^C,\, K_t^K,\, M_t,\, B_t,\, P_t,\, Z_t)$$

You can drop what you don't need; the minimal useful subset is:

- W_t: water budget (kL)
- L_t: labor hours budget
- A_t: "processing area / capacity" (e.g., barrel space, compost bays)
- K_t^N: KNF "N-equivalent credits" available
- K_t^C: carbon/compost/mulch credits available
- M_t: manure credits available (or split N/P/K)
- B_t: available forage/biomass (if grazing)
- P_t: parasite pressure (if ruminants)
- Z_t: stocking / head-days capacity remaining (optional)

Key idea: KNF and livestock are not "vibes"; they are *stocks and flows* that appear in the same balance as fertility and costs.

1.3 Full system state

Single bed:

$$X_t = (x_t, g_t)$$

Multiple beds:

$$X_t = \left(x_t^{(1)}, \ldots, x_t^{(B)},\, g_t\right)$$

---

2) Actions

At each step t and bed b:

2.1 Crop/guild action $c_t^{(b)} \in \mathcal{C}$

Each crop/guild has attributes:

- family: $fam(c)$
- revenue: $Rev_t(c)$ (may depend on season t)
- variable cost: $Cost_t(c)$
- resource demands: $w(c), \ell(c)$ etc.
- soil impacts: $\Delta_N(c), \Delta_C(c), \Delta_S(c)$
- nutrient demand vector (optional): $dem(c) = (n(c), p(c), k(c))$

You can interpret $\Delta_N(c)$ as nutrient removal, or as "net effect given typical residue return".

2.2 Input action $i_t \in \mathcal{I}$

Inputs are **production** and/or **application** operations, e.g.:

- produce KNF N: `make_KNF_N`
- apply KNF N to bed b: `apply_KNF_N(b, u)`
- compost production: `make_compost`
- compost application: `apply_compost(b, u)`
- purchased fertiliser: `buy_N(u)`
- irrigation event: `irrigate(u)` (or water is baked into crop demand)

Each input action consumes resources and adds to either farm stocks or bed soil:

- consumes: water/labor/space/money/biomass
- produces: K^N, K^C, etc., or directly increases N, C, S on a bed

2.3 Livestock action $\ell_t \in \mathcal{L}$

Examples:

- graze paddock/cover on bed b (ruminants)
- poultry tractor pass on bed b
- rest/recovery (no animals)
- cut-and-carry biomass into compost system

Livestock actions have:

- revenue: eggs/milk/meat value (or can be modelled separately)
- costs: labor, feed, water, fencing moves
- soil effects: manure adds N/C, but traffic can reduce S
- forage/manure flows: $\Delta B, \Delta M$
- parasite pressure updates: ΔP

---

3) Deterministic transition function

3.1 Bed transition

For each bed b:

$$x_{t+1}^{(b)} = T_{bed}\big(x_t^{(b)},\, c_t^{(b)},\, i_t,\, \ell_t,\, g_t\big)$$

In the simple additive bucket model:

$$\begin{aligned}
N' &= clamp(N + \Delta_N(c) + \Delta_N(i,\ell),\, 0..N_{\max}) \\
C' &= clamp(C + \Delta_C(c) + \Delta_C(i,\ell),\, 0..C_{\max}) \\
S' &= clamp(S + \Delta_S(c) + \Delta_S(i,\ell),\, 0..S_{\max}) \\
F' &= fam(c) \quad (\text{or unchanged if } c = \text{none}) \\
D' &= clamp(D + \Delta_D(c,F,D) + \Delta_D(\ell),\, 0..D_{\max})
\end{aligned}$$

A good deterministic disease rule is:

- if you repeat family too soon → D increases
- if you break family → D decreases
- livestock/poultry pass can decrease D for some pests but might increase for others; keep it coarse.

3.2 Global resource transition

$$g_{t+1} = T_{farm}(g_t, \{c_t^{(b)}\}_b, i_t, \ell_t)$$

Typical updates:

- $W' = W - \text{water used by crops and inputs}$
- $L' = L - \text{labor used}$
- $A' = A - \text{space occupied}$ (or capacity constraint per step)
- K^N increases when you *make* KNF inputs, decreases when you *apply* them
- B (forage) decreases when grazed, increases under rest/growth/cover crops
- M (manure credits) increases with grazing/stocking, decreases when applied/exported
- P (parasites) increases with grazing pressure, decreases with rest windows

Everything is deterministic: fixed deltas per action (or piecewise by state bucket).

---

4) Constraints (soil maintenance + feasibility)

4.1 Soil invariants (hard)

For all t and beds b:

$$N_t^{(b)} \geq N_{\min}, \quad C_t^{(b)} \geq C_{\min}, \quad S_t^{(b)} \geq S_{\min}$$

Optionally:

$$D_t^{(b)} \leq D_{\max}$$

4.2 Rotation constraints (hard)

Examples:

- No same family back-to-back:

$$fam(c_t^{(b)}) \neq F_t^{(b)}$$

- Cooldown of length k: keep last k families in state, forbid repeats (still deterministic; state includes a short history).

4.3 Resource constraints (hard)

Per step:

$$W_t \geq 0, \quad L_t \geq 0, \quad A_t \geq 0, \quad K_t^N \geq 0, \quad \ldots$$

Livestock-specific:

- forbid grazing if soil too wet/fragile:

$$\text{if } S_t^{(b)} \leq S_{wet} \text{ then grazing action disallowed}$$

(or use a separate "traffic risk" scalar if you want).

---

5) Reward / profit model

Per step t, total profit is:

$$R_t(X_t, a_t) = \sum_{b=1}^{B} \left(Rev_t(c_t^{(b)}) - Cost_t(c_t^{(b)}) \right) \; - \; Cost_t(i_t) \; - \; Cost_t(\ell_t)$$

Where costs include:

- labor valued at a wage (or opportunity cost)
- water at marginal cost
- materials (fish hydrolysate, sugars, minerals, bedding)
- depreciation if you want

Soil not just "maintained", but incentivized

If you only use hard minima, the optimizer tends to "hug the boundary". Add an end-of-year soil value:

$$R_{terminal}(X_{T+1}) = \lambda_N \sum_b N_{T+1}^{(b)} + \lambda_C \sum_b C_{T+1}^{(b)} + \lambda_S \sum_b S_{T+1}^{(b)}$$

This makes it prefer soil-positive plans when profit differences are small.

---

6) The optimization problem

Single bed (DP)

$$\max_{a_1,\ldots,a_T} \sum_{t=1}^{T} R_t(X_t, a_t) + R_{terminal}(X_{T+1})$$

subject to:

$$X_{t+1} = T(X_t, a_t), \quad X_t \in \text{Feasible } \forall t$$

Multiple beds with coupling (MIP recommended)

If beds share labor/water/KNF stocks, the optimal choice is a coupled optimization. DP can still work if you keep global state small; MIP scales better.

---

7) Solution methods

7.1 Deterministic Dynamic Programming (exact, easy for 1 bed)

Value function:

$$V_t(X_t) = \max_{a_t \in \mathcal{A}(X_t)} \left(R_t(X_t, a_t) + V_{t+1}(T(X_t, a_t)) \right)$$

with $V_{T+1} = R_{terminal}$.

Works great when state is bucketed and small.

7.2 Mixed Integer Programming (exact, best for many beds)

Binary decision variables like:

- $y_{t,b,c} \in \{0,1\}$: choose crop c on bed b at time t
- $u_{t,i} \geq 0$: amount of input action i at time t
- $z_{t,\ell} \in \{0,1\}$: livestock action choices

Then linear constraints enforce:

- one crop per bed per step
- resource budgets
- soil bucket transitions (piecewise linear or discretized)
- rotation rules
  Objective is linear: maximize revenue minus costs.

---

8) Data schema (what you must supply)

Crop/guild record

- `name`
- `family`
- `rev[t]` (or price × yield model)
- `cost[t]`
- `water[t]`, `labor[t]`
- `delta_soil: (ΔN, ΔC, ΔS, ΔD)` buckets
- optional: `nutrient_demand: (n,p,k)` physical units

Input record (KNF / compost / purchased)

- `name`
- `type: produce` or `apply` or `buy`
- resource consumption: labor, water, space, money, biomass
- stock changes: `ΔK^N`, `ΔK^C`, `ΔM`, etc.
- bed effect if applied: `ΔN,ΔC,ΔS`

Livestock record

- `name`
- revenue/cost per step
- forage consumption / manure production
- soil and disease impacts
- guards (e.g., minimum structure or maximum wetness)

Feasibility thresholds

- soil minima $(N_{\min}, C_{\min}, S_{\min})$
- disease max $D_{\max}$
- resource budgets per step $(W_t, L_t, A_t, \dots)$

---

9) Deterministic “dispatch” policy view (state machine)

The whole thing is a deterministic automaton with an optimal controller:

- **States:** X_t
- **Actions:** a_t

- **Transition:** $X_{t+1} = T(X_t, a_t)$
- **Accepting states:** Feasible (soil maintained, resources nonnegative)
- **Controller:** chooses a_t to maximize cumulative reward

So: **state machine + optimizer = most profitable feasible rotations**.

---

10) Minimal 1-year instantiation (what "complete" looks like in practice)

If you want the *smallest complete* instance that still includes crops + KNF + livestock:

- $T = 4$ (quarters)
- 10 crop actions (your palette)
- 3 input actions:
  - `make_KNF_N` (convert labor+water+space → K^N)
  - `apply_KNF_N(b,u)` (convert K^N → N on bed)
  - `apply_compost(b,u)` (convert K^C or purchased → C, S)
- 2 livestock actions:
  - `poultry_pass(b)` (adds N, reduces some pests, costs labor)
  - `rest` (forage regrowth / parasite break)
- State buckets $N, C, S \in 0..5$, F in 8 families

That is already "full formalism" and is implementable immediately.

If you want, I can now turn this into a **single self-contained spec file** (like a "model contract") with:

- explicit state vector definitions,
- action tables,
- transition equations,
- constraint definitions,
- and the DP recurrence,

so you can plug it into your codebase as the canonical formalism.

ok

Excellent. Below is your **single, canonical, self-contained formal specification** of the system.

This is written as a formal model contract — no handwaving, no narrative — suitable for implementation in DP, MIP, or as a simulation engine.

🌱 Integrated Farm Optimization Formalism

(Crops + KNF Inputs + Livestock + Soil Invariants)

---

1. Time

Discrete time horizon:

$$t \in \{1, 2, \ldots, T\}$$

For 1-year planning:

- $T = 4$ (quarters), or
- $T = 6$–8 (market garden windows)

---

2. Sets

- Beds: $b \in \{1, \ldots, B\}$
- Crop actions: $c \in \mathcal{C}$
- Input actions: $i \in \mathcal{I}$
- Livestock actions: $\ell \in \mathcal{L}$

Composite action per bed per time:

$$a_t^{(b)} = (c_t^{(b)}, i_t^{(b)}, \ell_t^{(b)})$$

Global actions (shared resources) allowed if needed.

---

3. State Space

3.1 Bed State

For each bed b:

$$x_t^{(b)} = (N_t^{(b)}, C_t^{(b)}, S_t^{(b)}, F_t^{(b)}, D_t^{(b)})$$

Where:

- N = fertility bucket (0…5)
- C = soil carbon bucket (0…5)
- S = structure bucket (0…5)
- F = last crop family (categorical)
- D = disease/pest pressure bucket (0…5)

Minimal version removes D.

3.2 Global Farm Resource State

$$g_t = (W_t, L_t, A_t, K_t^N, K_t^C, M_t, B_t, P_t)$$

Where:

- W_t = water available
- L_t = labor hours available
- A_t = input production capacity (space)
- K_t^N = KNF nitrogen credits
- K_t^C = compost/carbon credits
- M_t = manure credits
- B_t = forage biomass
- P_t = parasite pressure

Minimal subset:

$$(W, L, K^N, K^C, B)$$

3.3 Full System State

$$X_t = \left(x_t^{(1)}, \ldots, x_t^{(B)}, g_t\right)$$

---

4. Action Definitions

4.1 Crop Action c

Each crop $c \in \mathcal{C}$ defines:

- $fam(c)$
- Revenue $Rev_t(c)$
- Cost $Cost_t(c)$
- Resource use $w(c), \ell(c)$
- Soil deltas:

$$\Delta(c) = (\Delta_N, \Delta_C, \Delta_S, \Delta_D)$$

4.2 Input Action i

Each input action defines:

- Resource consumption:

$$(\Delta W, \Delta L, \Delta A)$$

- Stock production:

$$(\Delta K^N, \Delta K^C, \Delta M)$$

- Optional direct soil effects if applied.

Example:

- `make_KNF_N`：
  $\Delta L = -2,\ \Delta W = -1,\ \Delta K^N = +3$
- `apply_KNF_N(b,u)`：
  $K^N -= u,\ N^{(b)} += u$

4.3 Livestock Action ℓ

Defines:

- Revenue $Rev(\ell)$
- Costs $Cost(\ell)$
- Soil impact:

$$(\Delta_N^\ell, \Delta_C^\ell, \Delta_S^\ell)$$

- Biomass/manure flow:

$$\Delta B,\ \Delta M$$

- Parasite update:

$$\Delta P$$

Example:

- Poultry pass:
  - $\Delta_N = +1$
  - $\Delta_S = -1$ (if overworked)

- revenue from eggs
- labor cost

---

5. Deterministic Transition Function

5.1 Bed Transition

For each bed:

$$
\begin{aligned}
N' &= clamp(N + \Delta_N(c) + \Delta_N(i,\ell), 0, 5) \\
C' &= clamp(C + \Delta_C(c) + \Delta_C(i,\ell), 0, 5) \\
S' &= clamp(S + \Delta_S(c) + \Delta_S(i,\ell), 0, 5) \\
D' &= clamp(D + \Delta_D(c,\ell), 0, 5) \\
F' &= fam(c)
\end{aligned}
$$

All transitions are deterministic.

5.2 Global Resource Transition

$$g_{t+1} = g_t + \sum \Delta(\text{inputs}) + \sum \Delta(\text{livestock}) - \sum \Delta(\text{crop resource use})$$

Subject to:

$$W_t, L_t, A_t, K_t^N, K_t^C, B_t \geq 0$$

---

6. Constraints

6.1 Soil Invariants (Hard)

For all t, b:

$$N_t^{(b)} \geq N_{\min}$$

$$C_t^{(b)} \geq C_{\min}$$

$$S_t^{(b)} \geq S_{\min}$$

6.2 Rotation Constraint

No same family consecutively:

$$fam(c_t^{(b)}) \neq F_t^{(b)}$$

Extendable to k-period memory.

6.3 Livestock Guardrails

Example:

If $S_t^{(b)} \leq S_{wet}$, grazing disallowed.

If $P_t \geq P_{max}$, force rest action.

---

7. Objective Function

Total profit over horizon:

$$\max_{a_1,\ldots,a_T} \left(\sum_{t=1}^{T} R_t(X_t, a_t) + R_{terminal}(X_{T+1}) \right)$$

Where:

$$R_t = \sum_b \left(Rev(c_t^{(b)}) - Cost(c_t^{(b)}) \right) - Cost(i_t) - Cost(\ell_t)$$

Optional terminal soil value:

$$R_{terminal} = \lambda_N \sum_b N_{T+1}^{(b)} + \lambda_C \sum_b C_{T+1}^{(b)} + \lambda_S \sum_b S_{T+1}^{(b)}$$

---

8. Solution Methods

Single bed

Dynamic Programming:

$$V_t(X) = \max_{a \in \mathcal{A}(X)} \left(R_t(X, a) + V_{t+1}(T(X, a)) \right)$$

Multi-bed

Mixed Integer Programming (binary crop decisions + linear soil/resource updates).

---

9. Minimal Implementable Instance (1-Year, 4 Slots)

State buckets:

- $N, C, S \in \{0, \ldots, 5\}$
- Families: {Solanaceae, Brassica, Cucurbit, Legume, Allium}

Actions:

- 10 crop options
- 2 input actions (make/apply KNF)
- 1 livestock action (poultry pass)
- 1 rest action

Constraints:

- Soil ≥ (2,2,2)
- No family repeat

That system is:

- Fully deterministic
- Economically optimizing
- Soil-constrained
- KNF-aware
- Livestock-aware

---

10. Conceptual Summary

This formalism models the farm as:

Deterministic State Machine + Resource Balance System + Constrained Optimal Co

Crops extract.
KNF converts labor/biomass → fertility.
Livestock converts forage → manure + revenue.
Soil state enforces long-term viability.
Optimizer selects the profit-maximizing feasible trajectory.

If you would like, next I can:

- Convert this into executable Python DP code skeleton
- Convert it into a MIP model template
- Add stochastic weather layer
- Add price volatility modeling
- Or collapse it to a minimal 12-variable version for fast simulation

Your move.

README.md
File

What else can we add to the formalism from this?

Excellent — now we're at the right level.

You've already formalized:

- Soil dynamics
- Crop rotation
- KNF / input production
- Livestock nutrient cycling
- Profit optimization

From the **LES + Farm-to-Plate-and-Pond roadmap** in your README README , we can extend the formalism in a few *structural* ways that meaningfully increase power rather than just adding variables.

Below is what can be added to make this a **true coupled Living Farm Optimizer** rather than just a rotation planner.

🔁 1. Close the Nutrient Loop (Mass-Balance Extension)

Right now, soil fertility is bucketed. LES explicitly models:

- nutrient flows between water, soil, plants, animals README

So extend soil state from buckets to **explicit mass balance**:

Add State Variables

For each bed:

$$x_t^{(b)} = (N_t, P_t, K_t, C_t, S_t, F_t, D_t)$$

And for water bodies (aquaponics or pond):

w_t = (TAN_t, NO_2_t, NO_3_t, PO_4_t, DO_t)

Now crop uptake becomes:

$$N_{t+1} = N_t - U_N(c) + M_N(\ell, i)$$

Fish feed becomes:

$$TAN_{t+1} = TAN_t + f(\text{protein input}) - nitrification$$

Plants remove:

NO_3_{t+1} = NO_3_t - U_{NO3}(plant)

This integrates aquaponics nutrient cycling *directly* into crop fertility and removes the artificial separation between soil and water fertility.

🌧️ 2. Weather & Climate Forcing (LES Coupling)

LES includes:

- weather
- hydrology
- evapotranspiration 🗎 README

Add exogenous climate forcing:

$$\omega_t = (Rain_t, Temp_t, Solar_t, Wind_t)$$

Now crop yield becomes:

$$Yield(c, t) = Y_{base}(c) \cdot g(N_t, C_t, S_t, \omega_t)$$

Water budget:

$$W_{t+1} = W_t + Rain_t - ET_t - Irrigation_t$$

This turns the optimizer into:

> Optimal control under stochastic environmental forcing.

Now you can:

- simulate drought years
- evaluate irrigation investment
- test windbreak planting effect on evapotranspiration

🐄 3. Energy & Oxygen Budgets

Your roadmap includes automation and oxygen hooks.

Add:

For aquaculture:

$$DO_{t+1} = DO_t + Aeration_t - O_2_consumption(fish, microbes)$$

Constraint:

$$DO_t \geq DO_{critical}$$

Now feed formulation directly affects oxygen risk.
The optimizer may reduce protein % or increase aeration if electricity cost is lower.

This creates a real energy-nutrient tradeoff.

💰 4. Capital & Infrastructure State

LES supports management interventions.

Add infrastructure state:

$$I_t = (Greenhouse_area, Tank_volume, Aerator_capacity, Fencing_capacity)$$

Actions may include:

- build greenhouse
- add tank
- install aerator

State evolves:

$$I_{t+1} = I_t + Build_t$$

Add depreciation + capital constraint.

Now you can optimize:

- whether expanding tank volume is better than buying feed
- whether windbreak reduces irrigation cost enough to justify planting

This moves the model from “rotation optimizer” to **farm growth optimizer**.

📈 5. Risk & Uncertainty Layer

LES roadmap includes probabilistic reasoning.

Extend:

Let yield be stochastic:

$$Yield(c,t) = \bar{Y}(c,t) + \epsilon_t$$

Then solve:

- Expected profit maximization
- CVaR (Conditional Value at Risk)
- Robust optimization

Objective:

$$\max \mathbb{E}[Profit] - \rho \cdot Risk$$

Now the optimizer may choose diversified crops rather than high-margin monoculture.

🧠 6. Multi-Objective Optimization

Roadmap explicitly includes multi-objective optimisation README

Extend objective vector:

$$\max(Profit,\ SoilHealth,\ SelfSufficiency,\ GHG,\ Biodiversity)$$

Compute Pareto frontier via:

- NSGA-II
- ε-constraint method

This gives decision space instead of a single solution.

🐝 7. Biodiversity & Habitat Variable

LES includes wildlife & habitat modeling 🗎 README

Add habitat index:

$$H_t = f(Cover, Crop\ Diversity, Hedgerows)$$

Constraint or objective:

$$H_t \geq H_{min}$$

Livestock + crop choices influence biodiversity.

🌱 8. Succession / Long-Term Soil Carbon

Right now soil is bounded bucket.

Add long-term carbon dynamics:

$$C_{t+1} = C_t + inputs - respiration(C_t, Temp_t)$$

Respiration increases with temperature → climate sensitivity.

This allows evaluation of:

- cover crop frequency
- grazing intensity
- compost use
- reduced tillage

🔄 9. Cross-Domain Coupling (Diet ↔ Feed ↔ Farm)

From the roadmap:

- diet optimizer shares ingredient pool with feed optimizer 🗎 README

Add shared inventory:

$$Inventory_t = (Veg, Fish, Eggs, BSFL, Duckweed)$$

Human optimizer and feed optimizer both consume from:

$$Inventory_{t+1} = Inventory_t + Production_t - HumanUse_t - FeedUse_t$$

Now:

- eating more fish reduces feed demand
- feeding more BSFL reduces compost input
- planting more duckweed increases feed self-sufficiency

This is a **closed ecological-economical loop**.

⚡ 10. Automation Control Loop

Roadmap Phase 3 includes real-time adaptive control 🗎 README

Add:

$$Control_t = \pi(X_t)$$

Where policy π is learned (RL or MPC).

This converts the system into:

> Model Predictive Control over a living farm.

🔬 11. Emergent Property Metrics

Add emergent indicators:

- Nutrient circularity ratio
- % On-farm protein
- Energy return on energy invested (EROEI)
- Net ecosystem productivity
- Water use efficiency

These are functions of state trajectory:

$$Metric = \Phi(X_1, \ldots, X_T)$$

These allow evaluation beyond raw profit.

🧩 12. Formal Unification

The fully extended formalism becomes:

$$X_{t+1} = T(X_t, a_t, \omega_t)$$

$$a_t \in \mathcal{A}(X_t)$$

$$\max_{a_{1:T}} \sum_{t=1}^{T} R(X_t, a_t)$$

Subject to:

- Soil invariants
- Resource invariants
- Water & oxygen invariants
- Habitat invariants
- Capital constraints

Where:

$$X_t = (\text{Soil}, \text{Water}, \text{Livestock}, \text{Inventory}, \text{Infrastructure}, \text{Climate})$$

This is a **deterministic hybrid dynamical system with constrained optimal control**.

🏁 What This Becomes

You now have:

- Farm rotation optimizer
- Nutrient loop optimizer
- Feed & diet co-optimizer
- Environmental simulator
- Infrastructure planner
- Risk-aware controller
- Automation-ready control core

That is no longer a "rotation planner."

It is:

> A fully coupled agro-ecological optimal control system.

If you'd like, next we can:

1. Collapse this into a minimal but powerful 15-variable canonical core.
2. Separate "core deterministic engine" from "LES stochastic layer".
3. Write the full state transition in compact matrix form.
4. Or convert the whole thing into a formal control-theory document.

Your move.

Sources

WRite and run python which demonstrates based on the simpler verison from when you got prices

Thought for 14s

Here's a **running Python demo** of the *simple version* (the one with the 10-crop palette + the wholesale-ish prices we used earlier), solved as a **deterministic DP** for **1 bed, 1 year (4 seasons)** with **hard soil constraints** and a **no-same-family-back-to-back** rule.

It prints the optimal 4-step rotation, soil trajectory, and total profit under the toy yield/cost/soil-delta assumptions.

Output from the run (example result):

- Optimal plan (given the simple deltas + constraints) came out as:
1. **Tomato** → 2) **Cover crop** → 3) **Beans** → 4) **Tomato**
- Total profit: **$745.20 per bed-year** (toy numbers)
- End soil: **(N,C,S) = (2,3,3)** (still above minima)

You can now swap in your real yields/costs/KNF/livestock.

I ran it and got this plan table + summary:

text

```
   Season     Action      Family  Price(AUD/kg)  Yield(kg/bed)  Cost(AUD/b
0       1     Tomato  Solanaceae           3.98            120           
1       2  CoverCrop       Cover           0.00              0           
2       3      Beans      Legume           7.16             60           
3       4     Tomato  Solanaceae           3.98            100           

Summary:
Total profit (AUD): 745.2
End soil (N,C,S): (2, 3, 3)
Constraints: N,C,S >= (2,2,2); no same-family back-to-back (Cover resets).
```

Copy code

If you want, next I can extend this exact script in the smallest possible way to include:

- **KNF production/apply** as additional actions (resource-balanced),
- or a **poultry pass** action that trades labor for manure credits, while keeping the same DP structure.

KNF

Good — this is exactly what we want to see.

What happened?

With the current parameters:

- Producing KNF costs **$50** per batch
- Each batch produces **2 N credits**
- Applying KNF is optional

The optimizer chose:

Tomato → CoverCrop → Beans → Tomato

...and **never produced KNF**.

Why?

Because under the current soil model:

- The cover crop restores N cheaply.
- KNF costs more than simply inserting a cover crop.
- Therefore, KNF is economically dominated in this parameter regime.

🔎 This is the key insight

KNF only becomes optimal if:

$$(\text{Profit gain from avoiding cover crop}) > \text{KNF production cost}$$

Right now:

- Cover crop costs $70.
- KNF production costs $50 and only restores N.

- But cover crop restores N **and C and S**.

So cover crop is economically superior in this simplified model.

📈 How to Make KNF Become Optimal

If we change parameters such that:

- High-value crop (e.g. Tomato or Beans) is very profitable
- Cover crop opportunity cost is high
- KNF production is cheap (because labor is idle / inputs free)
- KNF restores only N but that is the binding constraint

Then the optimizer will switch.

For example:

- Reduce KNF cost to $20
- Increase tomato yield
- Reduce cover crop carbon benefit

Then the solver will prefer:

Tomato → Tomato (with KNF) → Beans → Tomato

instead of inserting cover crop.

🧠 What This Demonstrates

Your formalism is working correctly:

- KNF is just a resource conversion.
- It will be chosen **iff** it is economically superior to biological fertility pathways.
- It competes against:
  - cover cropping
  - livestock manure
  - purchased inputs

This is exactly how the system should behave.

If you'd like next, I can:

- Tune parameters so KNF becomes optimal and show the regime change.
- Add livestock manure as a competing N source.
- Add labor constraint so KNF production consumes limited labor.
- Or convert this into a multi-bed farm where KNF production scales across blocks.

Your move. ‹/›

Wouldn't knf be forced in long term?

Not automatically. In *this* formalism, **nothing is "forced"** unless you encode a constraint that makes it unavoidable.

What *is* true is: over long horizons, **some fertility source must close the nutrient/OM loop** (or you'll violate the soil invariants). KNF is one possible source, but it competes with others.

When KNF becomes "forced"

KNF (or purchased fert) gets forced if you have **a binding nutrient constraint** and the other recovery mechanisms can't meet it:

- **No/limited land or time for cover crops** (high cropping intensity, short windows)
- **No livestock / no imported manures/compost**
- **Residues exported** (you're selling biomass, not returning it)
- **Soil minima are hard** (e.g., $N \geq N_{\min}$ every period; or SOC can't drop)
- **Rotation rules restrict restorative crops**
- **Water limits** make biomass-building covers impractical

In that regime, the optimizer must choose *some* action that adds N (or offsets N drawdown), and if KNF is the only feasible one, it becomes forced.

When KNF is not forced (even long-term)

If any of these are available and cheaper, the model will prefer them:

- **Cover crops / green manures** (often restore N *and* C/S)
- **Livestock manure cycling** (forage → manure credits)
- **Compost / mulch imports** (or on-farm compost from residues)
- **Lower-intensity cropping** (fallow/rest windows)
- **Crop choice shift** toward legumes / lower-demand crops

In those cases, you still "maintain soil" without KNF; KNF becomes optional.

The deeper point: what's actually forced long-term

Long term, what's forced is **mass balance**, not a particular method:

- If you export nutrients in product every cycle, you must **replace** them via one of:
  - biological fixation (legumes / covers),
  - recycled manures/compost,
  - imported fertilizers,
  - or reduced exports (less harvest / more residue return).

KNF is best modeled as:

- a **conversion pipeline** that turns **on-farm biomass + labor + water** into **plant-available nutrient credits** (especially N/K/Ca depending what you're making), not as "free fertility from nowhere."

How to encode "KNF is forced" formally

If you *believe* KNF is operationally necessary for your system, add one (or more) of these constraints:

1. **Cropping intensity constraint**

$$\#\{t : c_t = \text{CoverCrop}\} \leq k$$

(you only allow ≤k cover slots per year)

2. **No external imports**

$$\text{PurchasedN}_t = 0, \quad \text{ImportedCompost}_t = 0$$

3. **Minimum cash-crop revenue**

$$\sum_t Rev(c_t) \geq R_{\min}$$

(you can't "solve" by doing mostly covers)

Then KNF (or livestock) may become the only feasible way to stay above $N_{\min}, C_{\min}, S_{\min}$.

What I'd do next in code

Two quick modifications will show you the regime shift immediately:

- Add a constraint: "at most 1 cover crop slot per year"
- Add a longer horizon: $T = 12$ (3 years of seasons)

Then re-run DP and you'll see whether the solver starts producing/applying KNF to keep N from falling below threshold.

If you want, I'll patch the demo to:

- run 3–5 years,
- cap cover-crop frequency,
- and show when KNF flips from "never used" to "used/forced."

assume organic only/onsite production only

Thought for 4 seconds

Yep — **"organic only + onsite production only"** is a very crisp set of constraints, and it changes what becomes "forced" over the long run.

The key punchline:

Something becomes forced, but it's not necessarily KNF.
What's forced is **nutrient/OM mass-balance closure using only onsite biological processes**. KNF is just one *pathway* inside that closure.

Here's how to encode it cleanly in the formalism.

---

1) Add explicit "onsite-only" constraints

Ban all external nutrient and carbon imports

For every time step t:

- Purchased / imported inputs are zero:

$$BuyN_t = BuyP_t = BuyK_t = 0, \quad ImportCompost_t = 0, \quad ImportManure_t = 0$$

- If you previously had an "input action set" $\mathcal{I}$, restrict it:

$$\mathcal{I} \leftarrow \mathcal{I}_{onsite}$$

where $\mathcal{I}_{onsite}$ includes only actions like:

- make/apply KNF inputs **from onsite biomass**
- composting **onsite residues/manures**

- grazing **onsite forage**
- cover crops / green manures
- residue mulching / incorporation

Enforce "feed is onsite" if you have livestock

If livestock exist and you want onsite-only feed:

$$FeedImport_t = 0$$

and require consumption ≤ onsite forage/biomass stock:

$$FeedUse_t \leq B_t$$

This is a huge constraint: it makes livestock not a "nutrient source" unless you're cycling **your own** biomass.

---

2) Promote nutrients from "buckets" to a required balance (minimal version)

Under onsite-only, the model needs to respect a simple reality:

> You can't replace exported nutrients with "process" alone; you need **a source term** inside the farm boundary.

The only onsite "source" for **new N** (ignoring deposition) is:

- **biological N fixation** (legumes, some free-living microbes), possibly enhanced by management

So add an explicit **N budget** per bed:

$$N_{t+1} = N_t - ExportN(c_t) + FixN(cover/legume_t) + RecycleN(residues, manure, KNF)$$

Where:

- **ExportN** is proportional to yield harvested and sold.
- **FixN** is positive only for legume/green manure actions.
- **RecycleN** just moves N around; it cannot exceed what was already onsite.

Minimal-on-purpose way to implement this without full chemistry

Keep your bucket model, but enforce:

- Every cash crop has an **N export** term
- Only legume covers have an **N input** term
- KNF has **no net creation**, only conversion + improved availability (see below)

---

3) What KNF means under onsite-only

Under onsite-only, KNF cannot be modeled as "create N."

Instead, KNF is one (or both) of:

A) Availability shift / loss reduction

KNF increases effective uptake, reduces volatilization/leaching, etc.:

- Replace crop yield response function:

$$Yield = Y_{base} \cdot g(N_{avail})$$

with KNF increasing N_{avail} given same underlying N.

B) Faster cycling (time shift)

KNF converts “slow” nutrients in biomass/manure into “available” form quicker.

This means your state should split N into at least two pools:

- N^{slow} (in residues/biomass/manure/soil organic pool)
- N^{avail} (plant-available)

Then KNF action performs:

$$(N^{avail}, N^{slow}) \mapsto (N^{avail} + \delta,\ N^{slow} - \delta)$$

with $\delta \leq N^{slow}$.

That’s how KNF becomes valuable without violating mass balance.

---

4) Under these constraints, what becomes forced long-term?

Forced #1: Some legume/green-manure (or N-fixing component)

If you continuously export product, and you disallow imports, then without N fixation:

- N monotonically declines
- eventually you violate $N \geq N_{\min}$

So the system *must* include a positive-N action periodically (legume cover, ley, etc.) unless you also allow reducing exports dramatically.

Forced #2: Carbon/OM maintenance via residues + covers

SOC/structure is also forced because:

- heavy harvesting + disturbance tends to reduce C, S
- imports are banned

So you must allocate time/biomass to:

- cover crops
- mulching/incorporation
- composting/manure cycling (from onsite biomass)

KNF is forced only if you add a further constraint

KNF becomes *forced* only if you also require:

- **high cropping intensity** (very limited cover slots), *and*
- **high export targets**, *and*
- **no livestock** (or livestock constrained), *and*
- tight soil minima

Formally, something like:

$$\#\{t : c_t \in \text{legume cover states}\} \leq k$$

plus

$$\sum_t Revenue(c_t) \geq R_{\min}$$

plus onsite-only

Then the optimizer may be forced to use KNF to keep N^{avail} high enough between fixation windows.

5) Minimal changes to our state machine to support onsite-only correctly

If you want the *smallest* upgrade that makes this honest:

Upgrade soil state

From:

- N bucket

To:

- N^{avail}, N^{slow} (two buckets)
- C, S as before

Redefine action deltas

- Cash crops: export from N^{avail} (and some from N^{slow} via residue removal)
- Covers (legume): add to N^{slow} via fixation + biomass
- Compost/manure: add to N^{slow}
- KNF: move $N^{slow} \to N^{avail}$ (conversion), costs labor/water

This automatically produces the behavior you're intuiting:

- long-term, you must fix N somewhere
- KNF can become economically optimal to keep production high in the "in-between" periods

---

If you want, I can patch the demo code to enforce onsite-only and make KNF "conversion only" (two-pool N). It will show very clearly:

- in which regimes KNF is never used,
- used occasionally,
- or becomes unavoidable given your cropping-intensity/export constraints.

consider integrating livestock now, also could we consider an 'optimal soil builder' rotation? totake shit soil and make it good? like soil very bad, no or no water/nutriend import nutrient available, how to profitably crop (real answer is likely perennials included but maybe too much scope increase at once)

Yes — and this is exactly the moment where the formalism stops being "rotation with constraints" and becomes a **two-mode optimal control problem**:

1. **Rehab mode:** take *bad soil → good soil* under onsite-only constraints
2. **Production mode:** once soil is above targets, maximize profit while maintaining

Livestock integrates as the "biomass concentrator + revenue during rehab" layer.

Below is how to integrate both **without exploding scope**, and a realistic "optimal soil builder rotation" you can compute.

---

1) Livestock integration under onsite-only

The honest constraint

Under **onsite-only**, livestock do **not create nutrients**. They:

- **concentrate** nutrients spatially (manure where you want it),
- **accelerate cycling** (biomass → manure/compost → available),
- **create revenue** while you run cover/rehab phases,
- but only from **onsite biomass** (unless you explicitly allow importing feed/bedding).

So in the model:

- Livestock actions **convert** `Biomass (B)` into `Manure credits (M)` + product revenue, with losses and compaction risk.

Minimal state additions (enough to behave correctly)

Per bed/paddock (or per "block" if you don't want spatial detail yet):

- `B` = standing biomass/forage (0..5)
- `M` = manure/compostable nutrient stock on-farm (0..5)
- `S` already exists (structure), but add:
  - `T` = traffic/wetness risk (0..5) *or* derive from season + soil type
- optional: `P` parasite pressure if ruminants

Livestock actions (minimal set)

- `GRAZE_LIGHT` : B↓, M↑, revenue+, S slight↓ only if T high
- `GRAZE_HARD` : more revenue/processing but higher S risk
- `REST_REGROW` : B↑ (growth), P↓ if modeled
- `BEDDED_PACK` : convert collected bedding+manure to "C credits" (if bedding is onsite straw/woodchip)

This is enough to capture the central trade: **livestock can fund soil rebuilding but can also wreck structure if mistimed**.

---

2) "Optimal soil builder" as a first-class objective

Right now your objective is profit with soil constraints. For rehab you want *the reverse*:

Define a target soil state

Let soil be `(N, C, S)` buckets (or two-pool N later).
Define target thresholds (good soil):

- $N \geq N^*$
- $C \geq C^*$
- $S \geq S^*$

Define a soil quality function

A simple scalar:

$$Q(x) = w_N N + w_C C + w_S S$$

or a minimum-distance-to-target:

$$d(x) = \max(0, N^* - N) + \max(0, C^* - C) + \max(0, S^* - S)$$

Rehab optimization problem

Two good ways to formalize “take shit soil and make it good”:

A) Minimum time to reach target + profit tie-break

$$\min \ \tau \quad \text{s.t. } X_\tau \in \mathcal{G}$$

and among equal τ, maximize profit.

B) Maximize profit subject to achieving target by deadline

$$\max \sum_{t=1}^{T} Profit_t \quad \text{s.t. } X_T \in \mathcal{G}$$

This stops the model from “hovering near bad forever because covers cost money” — it forces actual rehabilitation.

---

3) The hard reality: onsite-only + bad soil means you can’t “fix it with money”

If you ban imports of:

- nutrients,
- compost/manure,
- feed,
- (and maybe even bedding),
  then your only true “source terms” are:

Nitrogen source

- **biological N fixation** (legume covers / ley)

Carbon source

- **photosynthesis → biomass** (covers, perennials, residue return)

Structure source

- root systems + biology + reduced disturbance; deep roots help but still need water.

So the model will “discover” the same thing farmers know:

- In very degraded soil, you must allocate time to **non-cash biomass-building**.
- Livestock is how you can still earn something while doing that.

---

4) A realistic rehab rotation that stays “small scope”

Keep annual-only for now. Use these archetypes:

Crop/cover archetypes (actions)

1. `LEGUME_LEY` (vetch/clover/medics) → **adds N_slow and C**
2. `CARBON_PUMP` (rye/oats/sorghum) → **adds C and S**
3. `DIVERSITY_MIX` (multi-species cover) → modest all-around, improves resilience
4. `CASH_LOW_INPUT` (beans / hardy alliums) → revenue, modest drawdown
5. `CASH_HIGH_VALUE` (tomato etc.) → revenue, heavy drawdown (usually disallowed early rehab)

Livestock archetypes (actions)

6. `GRAZE_LIGHT`
7. `REST_REGROW`

KNF archetype (optional, onsite-only honest meaning)

8. `KNF_CATALYZE` : converts `N_slow → N_avail` (and/or reduces loss), costs labor/water

This is still manageable.

---

5) What "optimal rehab" often looks like (qualitative policy)

Assume initial soil is bad:

- `(N,C,S) = (0,0,0)` or `(1,1,1)` and onsite-only, water limited.

A profit-seeking rehab policy usually goes:

Phase 1: Establish biomass engine (you can't graze nothing)

- `REST_REGROW` / plant a hardy cover that will grow with minimal inputs
- avoid tillage (don't spend structure you don't have)

Phase 2: Run a legume + grass cycle (the true engine)

- alternate `LEGUME_LEY` and `CARBON_PUMP` (or mix them)
- if livestock present: `GRAZE_LIGHT` intermittently to get revenue + manure concentration
- keep grazing conservative if structure is low or wet

Phase 3: Introduce "cash-light" crops while maintaining covers

- `CASH_LOW_INPUT` slots (beans, some alliums, hardy brassicas) *only when N/C/S allow*
- still keep frequent covers

Phase 4: Transition to production rotations

- once above targets, allow tomato/high-value states, supported by onsite fertility loop

This is exactly what your optimizer will find once you put in the right objective (reach target) and onsite-only constraints.

---

6) Minimal formal change to support "bad soil rehab" correctly

Right now, your `N` bucket is "available fertility." In rehab, the difference between *available* and *stored* matters a lot.

So the single most valuable minimal upgrade is:

Split nitrogen into two pools

- N^{slow}: stored in biomass/OM/manure
- N^{avail}: plant-available

Then:

- Covers increase N^{slow} (and sometimes avail slightly).
- Cash crops consume N^{avail}.
- KNF and composting **convert** $N^{slow} \to N^{avail}$.

That makes “KNF as catalyst” real, and makes livestock/manure act on the right pool.

---

7) If you want a “compute-able” rehab answer next

The next step is to pick:

- rehab horizon T (e.g., 8 seasons = 2 years),
- soil target (N^*, C^*, S^*),
- water/labor limits,
- whether livestock exists and what it can eat (onsite biomass only),
- and which crops you allow in rehab.

Then we can compute:

- **minimum-time to target**, and
- **most profitable path that hits target by time T**.

No perennials needed yet; we can add them later as “long-duration actions” once the annual rehab policy is stable.

If you want, I’ll write the Python DP that explicitly does *rehab mode* (min time to reach targets) with livestock grazing as a revenue+manure conversion, staying onsite-only.

So now I guess the optimiser is soil-replenishment-respecting export that maximises selection factor (import vs export I guess... what sort of eigens would you get here?)

Yeah — once you make it **onsite-only** and **soil-replenishment-respecting**, the optimizer is basically finding **steady nutrient/biomass flow modes**. “Eigens” are the right intuition, but the right object is:

- **fixed points / limit cycles** of the **state transition under an optimal policy**, and/or
- **eigenvectors of a linearized nutrient-flow operator** around those cycles.

Here’s the clean way to think about it.

---

1) What’s being optimized now?

With onsite-only, every season you choose actions that:

- **export** nutrients/carbon in products (crops/eggs/meat)
- **replenish** via internal processes:
  - N fixation (legumes) is the *only true source of new N*
  - photosynthesis is the *source of new C/biomass*
  - livestock/KNF shift nutrients between pools (availability + spatial concentration), but don’t create mass

So the feasible set is the set of action sequences where:

$$\Delta \text{Soil} \geq 0 \quad (\text{or at least } \geq \text{minima})$$

while still exporting product.

Your “selection factor” is basically:

$$\text{selection} = \text{export value} - \text{internal cost} - \text{risk/constraints penalties}$$

---

2) The “operator” and its eigens

Let the state be (simplified):

$$x_t = (N_t^{slow}, N_t^{avail}, C_t, S_t, B_t, M_t)$$

And an action a_t (crop + grazing + KNF) defines a deterministic map:

$$x_{t+1} = T_{a_t}(x_t)$$

Policy induces a single operator

An optimal policy π produces:

$$x_{t+1} = T_\pi(x_t) := T_{\pi(x_t)}(x_t)$$

Now you can define “eigens” in two related ways:

A) Fixed points (true eigens in the nonlinear sense)

A fixed point is:

$$x^* = T_\pi(x^*)$$

Interpretation: a soil + biomass + manure equilibrium where your management choices keep you in the same state each cycle.

B) Limit cycles (periodic eigens)

Most farms are seasonal, so you get:

$$x_{t+k} = x_t$$

A k-cycle corresponds to a repeating rotation that maintains soil exactly (or in a bounded band).

C) Linearized eigenmodes (local stability, contraction/expansion)

Linearize around a fixed point or a point on the cycle:

$$\delta x_{t+1} \approx J\, \delta x_t$$

where J is the Jacobian of T_π.

Eigenvalues of J:

- $|\lambda| < 1$: perturbations decay → stable mode
- $|\lambda| > 1$: perturbations grow → unstable mode (system collapses unless constraints intervene)
- $|\lambda| \approx 1$: neutral directions (“marginal” constraints you ride along)

Those are literal “eigens”.

---

3) What eigens do you actually get in onsite-only agriculture?

You tend to see a small number of **attractor archetypes** (eigen-cycles), because the constraints force them.

Eigen-cycle 1: N-fixation–dominated loop (legume ley → cash crop)

- Legume phase builds N^{slow}
- Cash phase converts $N^{slow} \to N^{avail}$ (via mineralization/KNF/manure)
- Exports some N in product
- Returns residues to keep C from collapsing

This often shows up as a **2–4 step cycle**.

Eigen-cycle 2: Carbon/structure–dominated loop (carbon pump → graze → compost/manure → cash-light)

If C/S are binding constraints (degraded soil), the stable cycle is:

- carbon pump cover (grasses) → graze lightly → rest/regrow
- occasional cash-light crop
- compost/manure returns prevent C/S loss

This is the “rehab eigen”.

Eigen-cycle 3: Livestock-mediated equilibrium

Where livestock + forage become the main engine:

- forage → animal product + manure → soil → forage

Your exports are:

- eggs/meat/milk
- possibly some crops

This can be very stable (strong negative feedback) if grazing is controlled; unstable if traffic/overgrazing enters (structure collapses).

Eigen-cycle 4: High-value crop with catalytic fertility

This only exists if you have enough internal cycling capacity:

- high-value crop exports hard
- KNF + composting + legumes keep N^{avail} from crashing
- requires enough biomass/water/labor to maintain cycling

It’s a “thin” attractor: high profit, but narrow feasible band.

---

4) “Import vs export” as an eigenvalue problem

If you allow imports, the system has an external forcing term u_t:

$$x_{t+1} = T(x_t, a_t) + u_t$$

Onsite-only sets $u_t = 0$, so sustainability is strictly about whether your internal operator has a **bounded invariant set**.

A nice summary statistic is a **nutrient reproduction ratio** (per cycle):

$$R_N = \frac{\text{N fixed} + \text{recycled to avail}}{\text{N exported} + \text{lost}}$$

- $R_N > 1$: soil N climbs (rehab regime)
- $R_N = 1$: sustainable steady mode (true eigen-cycle)
- $R_N < 1$: eventual collapse unless you reduce exports

Same for carbon:

$$R_C = \frac{\text{C inputs (biomass returned)}}{\text{C losses (respiration + disturbance + export)}}$$

The "eigens" correspond to action cycles where $R_N \approx 1$ and $R_C \approx 1$ (with S stabilized).

---

5) How to compute the eigens in practice (with our DP)

Two straightforward ways:

Method 1: Find repeating optimal rotations (limit cycles)

Run the optimizer for a long horizon (say 20–40 seasons) with onsite-only and soil constraints, and look for periodic repetition in the chosen action sequence and state.

That reveals the "eigen-cycles".

Method 2: Compute a best "sustainable cycle" directly

Enumerate candidate k-cycles of actions (k=2..6), simulate the state transition over one cycle, and keep those where:

- soil returns to same or better
- resources nonnegative
- maximize average profit per season

This gives you the **principal eigen-cycle** under your constraints.

Then you can linearize around it to get stability eigenvalues.

---

6) What to expect conceptually

Under onsite-only:

- The optimizer's top solutions are almost always **limit cycles**, not fixed points.
- You'll see a small set of cycles:
  - rehab cycle (build C/S)
  - sustainable production cycle (N-fix + cash)
  - livestock-centered cycle (forage → manure → forage)

Your "selection factor" is basically which cycle has the best **profit per unit soil reproduction ratio** while staying stable.

If you want, I can modify the demo code to:

1. split N into **slow/avail**,
2. add a simple **grazing action** (biomass→manure→N_slow),
3. run 10 years,
4. detect the emergent **periodic eigen-cycles**, and print the principal one (max average profit while exactly sustainable).

soil-optimal restorative while profitable

Yes. Formally, that's a **bi-criterion optimal control** problem:

- **Restore soil as fast/strong as possible**
- **Stay profitable (or at least non-negative cashflow)**

There are two clean (and practical) ways to define "soil-optimal restorative while profitable".

1) The cleanest definition: maximize soil restoration subject to a profit floor

Let soil state for a bed be $x_t = (N_t, C_t, S_t, \dots)$. Define a soil quality score (pick weights you care about):

$$Q(x) = w_N N + w_C C + w_S S$$

Define the total soil gain over the horizon:

$$\Delta Q = Q(x_T) - Q(x_0)$$

Define total profit:

$$\Pi = \sum_{t=1}^{T} \text{profit}(a_t)$$

Now solve:

$$\max_{a_{1:T}} \ \Delta Q \quad \text{s.t.} \quad \Pi \geq \Pi_{\min}, \ \text{onsite-only constraints}, \ \text{feasibility}$$

Interpretation:

- You get the **most restorative plan possible** that still meets your minimum cash requirement.
- Set $\Pi_{\min} = 0$ for "profitable (non-negative)".

This avoids the common failure mode where the optimizer sacrifices soil to chase profit, or sacrifices profit to do only covers.

What this yields in practice

It tends to pick:

- lots of **biomass-building + N-fixing** phases early (rehab)
- plus the **highest "profit per soil damage"** cash crops occasionally
- livestock grazing is chosen if it increases ΔQ per unit of forgone profit (because it lets covers generate revenue while still building soil)

---

2) The practical engineering definition: maximize profit with a soil-restoration target

Flip it:

$$\max_{a_{1:T}} \ \Pi \quad \text{s.t.} \quad Q(x_T) \geq Q^*, \ \text{onsite-only}, \ \text{feasibility}$$

Interpretation:

- "Make soil good by year-end, and within that, make the most money."

This is how you run a rehab plan with a deadline.

---

3) "Eigen" view of restorative profitable plans

Under onsite-only, the "good" solutions are usually **limit cycles** (rotations) that:

- have **net positive** ΔQ until you hit target soil,
- then settle into an **approximately neutral** sustainable cycle ($\Delta Q \approx 0$) with maximal average profit.

So the "soil-optimal but profitable" mode often looks like:

- a transient "rehab trajectory"
- then a steady "production eigen-cycle".

---

4) The smallest model tweak that makes this behave correctly

If you're serious about restoration, split nitrogen into two pools:

- N^{slow}: stored in biomass/OM/manure
- N^{avail}: plant-available

Then restorative actions increase N^{slow}, and KNF/livestock/compost shift $N^{slow} \to N^{avail}$.

This prevents the optimizer from "cheating" by pretending it can conjure plant-available fertility without building the underlying pool.

---

5) What "restorative while profitable" usually selects (rule-of-thumb)

It selects actions with the best ratio:

$$\text{Restoration Efficiency} = \frac{\Delta Q}{\text{Profit loss vs best cash crop}}$$

Typical winners:

- **legume + grass multi-species cover** (big $\Delta C, \Delta S$ plus some N^{slow})
- **light grazing** on covers (revenue + manure concentration, small structure penalty if timed right)
- **cash-light legumes/alliums** as "bridge cash crops"
- avoid high disturbance harvests (potatoes) during early rehab unless margins are exceptional

---

If you want, I can modify the exact DP demo you already have to compute:

- the **most restorative plan with** $\Pi \geq 0$, and
- the **Pareto frontier** (profit vs soil gain) for your 10-crop palette (plus optional livestock action),

so you can literally see the set of "soil-optimal while profitable" solutions and pick your preferred point.

Carbon farming is becoming very popular in australia, what can we add here?

run these for a few of the systems we discussed...

Method 1: Find repeating optimal rotations (limit cycles)

Run the optimizer for a long horizon (say 20–40 seasons) with onsite-only and soil constraints, and look for periodic repetition in the chosen action sequence and state.

That reveals the “eigen-cycles”.

Method 2: Compute a best “sustainable cycle” directly

Enumerate candidate k-cycles of actions (k=2..6), simulate the state transition over one cycle, and keep those where:

soil returns to same or better

resources nonnegative

maximize average profit per season

This gives you the principal eigen-cycle under your constraints.

Thought for 1m 10s

Thought for 3m 44s

Here’s what I’d add for **Australian carbon farming**, and I **ran both methods** (limit-cycle detection + best sustainable k-cycle) on **4 systems**:

- Crops only
- Crops + livestock
- Crops only + carbon revenue
- Crops + livestock + carbon revenue

What to add (AU carbon farming reality)

To make your optimiser “carbon-farming aware” in Australia, the big additions are:

1) A carbon-credit reward channel tied to *measured/modelled SOC increase*
The ACCU soil carbon method explicitly credits **increases in soil organic carbon** from “new or materially different” management activities (measurement + models approach). Clean E... +1

2) Permanence obligations + discounting
If you choose a **25-year permanence** period, there’s a **20% discount** on ACCUs issued (the scheme also has other permanence options). Clean Energy Re...

3) Sampling/assurance costs + risk of invalid baseline
Baseline soil sampling is required; CER guidance emphasizes compliance and the importance of valid baseline sampling. Clean Energy Re... +1

(So, in the optimiser: carbon revenue should be net of an amortized "MRV cost" and include an invalid-baseline risk penalty if you want realism.)

4) ACCU price signal (for revenue conversion)
For the demo, I used a recent spot close of **A$36.90** (Core Markets, updated to end of day 18/02/2026). coremarkets.co
(You can also add a "price cap / reserve" perspective via the Safeguard cost containment measure price as an upper bound reference in scenarios.) Clean Energy Re...

5) Multi-activity bundling trend (incoming IFLM method)
Australia is actively working on an **Integrated Farm and Land Management (IFLM)** method intended to allow multiple carbon-storing activities on one property, streamlining participation. dcceew.gov.au +1
This maps perfectly to your "guild / rotation / livestock / biomass / compost / cover" combined state machine.

Results: the "eigen-cycles" we found

Method 1 (long horizon, 40 seasons): limit-cycle detection

With the current toy parameters, **no short (2–6) repeating tail cycle** was detected in the last 12 steps for any of the 4 systems (the policy keeps making small state-dependent adjustments).
That's normal in a finite-horizon DP with terminal soil value: it often doesn't settle into a clean periodic orbit.

If you want it to "snap" into eigen-cycles more reliably, we can do either:

- infinite-horizon **average reward** value iteration, or
- add explicit "rotation repeat bonus" / "steady-state preference" to the objective.

Method 2 (best sustainable k-cycle, k=2..6): principal sustainable cycle

This one *did* find a clean sustainable cycle for all 4 systems:

Best sustainable cycle (from start state N=C=S=3, biomass B=2)
Cycle length **k=3**:

> **Beans → CoverCrop → Beans**

It's sustainable because it returns soil **>= start** (actually builds N/C/S in this bucket model) and keeps biomass nonnegative.

When carbon revenue is enabled, the same cycle remains optimal but **CoverCrop becomes less costly** because it earns carbon revenue (in the toy mapping).

In the run, the cycle tables looked like:

- Without carbon: CoverCrop step value = **–70.00**
- With carbon: CoverCrop step value = **–46.38** including **~23.62** carbon revenue (toy conversion)

(Those numbers are exactly what you'd expect: carbon credits make restorative phases less financially painful.)

What carbon farming adds to your formalism (concretely)

Add a carbon ledger

Add to state:

- C_{soc} (or keep your C bucket but make it explicitly SOC-like)
- optional: crediting buffer / uncertainty budget

Add to reward:

$$R_t = \text{profit}_t \; + \; P_{ACCU} \cdot \Delta CO2e_t \cdot (1 - \text{permanence discount}) \; - \; \text{MRV amortized cost}$$

Add MRV constraints

- baseline sampling required; periodic re-measurement; method compliance gates

  Clean Energy Re... +1

  So a “carbon project on/off” flag can gate whether you can claim carbon revenue at all.

Add “additionality / practice change” flags

The soil carbon method credits increases from “new/materially different” practices

Clean Energy Re...

So your state machine should track whether management changes are “eligible” vs business-as-usual.

Next step (fast, and it will show real eigens)

If you want the system to *visibly* settle into eigen-cycles under Method 1, tell me which you prefer:

1. **Infinite-horizon average reward** (my recommendation)
2. Keep finite-horizon but add a “repeat-cycle preference” term

Either way, we’ll get clean “principal eigen-rotations” for:

- crops-only
- crops+livestock
- carbon-on vs carbon-off
- rehab mode vs production mode

...and you’ll see the optimiser shift from “profit only” to “profit + carbon + soil reproduction”.

If you want, I’ll implement (1) directly on top of the code we just ran and output:

- the top 3 eigen-cycles
- their profit/season
- their net soil drift
- their net carbon revenue/season (using ACCU spot) coremarkets.co +2

‹/›

Sources

i'm just worried infinite horizon will give an inaccurate picture but it's a useful test case in any event.. go ahead

Done — I ran an **infinite-horizon average-reward** solver (Relative Value Iteration) for the 4 systems, and then **simulated the greedy policy until it hit a true limit cycle** ("eigen-cycle").

Heads-up (important)

With the current toy model + hard soil minima, the **RVI did not numerically converge** within the iteration budget (diff stayed large), but **the induced greedy policy still settles into a clean, repeating cycle** when simulated. That's the practical thing you asked for: the *observed* eigen-cycle.

In other words: **even if the value function isn't converged, the long-run policy behavior is stable enough to reveal the repeating rotation.**

Infinite-horizon results (average reward + observed eigen-cycle)

All 4 systems produced the **same 12-step eigen-cycle**, with carbon-on simply increasing the per-step average reward.

Summary (per season-step)

- **CropsOnly:** average ≈ **158.10 AUD/step**
- **Crops+Livestock:** average ≈ **158.10 AUD/step** (livestock not used in the eigen-cycle under these toy parameters)
- **CropsOnly+Carbon:** average ≈ **165.97 AUD/step**
- **Crops+Livestock+Carbon:** average ≈ **165.97 AUD/step**

Eigen-cycle (12-step rotation) (same in all systems):

> **Tomato** → **Cucumber** → **Beans** → **CoverCrop** → (repeat through the seasons, forming a 12-step loop)

This is exactly the kind of "principal eigen-rotation" you were aiming at: it mixes

- **high value export** (tomato/cucumber),
- **cash-legume** (beans),
- **restoration** (cover crop)
  to keep the soil buckets above constraint thresholds indefinitely.

Why livestock didn't appear (yet)

In *this* toy parameterization, grazing:

- consumes biomass,
- gives modest revenue,

- and hits structure (S) a bit,

...so it's not competitive versus just running the crop/cover loop.

If you change *any* of these to better match your real situation (which I expect you will):

- higher grazing revenue,
- lower compaction penalty when timed right,
- biomass accumulation being more realistic,
- or tighter labor limits,

...then **livestock will show up in the eigen-cycle** quickly.

Where the tables are

In the notebook output I displayed:

- **"Infinite-horizon — summary"** (the 4-system comparison)
- **one table per system** labeled **"Eigen-cycle — <system>"** showing each step's:
  - action
  - reward
  - state and next_state

Next upgrades that make this "accurate enough to trust"

To address your worry about infinite-horizon accuracy (which is valid), the safest next step is:

1. **Extract the candidate eigen-cycle(s)** from this infinite-horizon run (done)
2. **Evaluate them under finite-horizon / scenario simulation**:
   - drought years, price shocks, pest pressure, etc.
3. Compare to a finite-horizon DP optimum as a check.

That way, infinite-horizon is used exactly as you intended: a **useful test case** that finds "principal modes," then you validate them.

If you want, I can also:

- enumerate the **top 5 eigen-cycles** (not just the one reached from the default start state),
- compute each cycle's **net soil drift per cycle** and **profit per cycle**,
- and show which ones dominate under carbon-on vs carbon-off.

1,2,3

Thought for 13s

Thought for 12s

1) Top Cycles (Season-Aware, Exact Return) — CropsOnly

Unable to display visualization

2) Scenario Eval (10y MC) — CropsOnly

Unable to display visualization

1) Top Cycles (Season-Aware, Exact Return) — Crops+Livestock

Unable to display visualization

2) Scenario Eval (10y MC) — Crops+Livestock

Unable to display visualization

1) Top Cycles (Season-Aware, Exact Return) — CropsOnly+Carbon

Unable to display visualization

2) Scenario Eval (10y MC) — CropsOnly+Carbon

Unable to display visualization

1) Top Cycles (Season-Aware, Exact Return) — Crops+Livestock+Carbon

Unable to display visualization

2) Scenario Eval (10y MC) — Crops+Livestock+Carbon

Unable to display visualization

Got it — I ran **(1) cycle extraction**, **(2) 10-year scenario Monte Carlo evaluation**, and **(3) finite-horizon DP comparisons** for the **4 systems**.

You should now see a bunch of tables in the notebook output:

- **"1) Top cycles ..."** (top 5 season-aware exact-return cycles, lengths 4/8/12)
- **"2) Scenario eval ..."** (10y MC: mean + p10/p50/p90 + mean soil drift)
- **"3) Finite DP ..."** (first 12 actions of the optimal 10-year plan, baseline + expected-scenario proxy)
- **"Overview ..."** (totals per system)

Quick read of the key outcomes

3) Finite-horizon DP totals (10y / 40 steps)

From the overview table:

- **CropsOnly (no carbon):** baseline total ≈ **6339.40**
- **Crops+Livestock (no carbon):** baseline total ≈ **6339.40** (livestock not chosen in the optimum under current toy params)
- **CropsOnly + Carbon:** baseline total ≈ **6646.41**
- **Crops+Livestock + Carbon:** baseline total ≈ **6646.41**

Expected-scenario proxy (adds drought probability + volatility in expectation, conservative soil transition):

- **CropsOnly:** ≈ **3608.69**
- **Crops+Livestock:** ≈ **3678.69**
- **CropsOnly + Carbon:** ≈ **4033.78**
- **Crops+Livestock + Carbon:** ≈ **4082.53**

So: **carbon revenue increases the optimum**, and under "robust expectation," **livestock+carbon starts to help slightly** (because it provides a less yield-sensitive revenue stream in this toy setup).

1) Top "eigen-cycle candidates"

These are **exact-return, season-aware** cycles (they return to the exact same state including season index). They're your cleanest "principal modes" for the onsite-only + soil-minima regime.

The optimizer is basically selecting cycles that mix:

- **high-margin export crops**
- **a legume / N-stabilizer**
- **a cover / C+S stabilizer**
  and (in some regimes) livestock/rest.

2) Scenario evaluation (10y Monte Carlo)

For each cycle candidate, I simulated 10 years with:

- price volatility (~10%)
- yield volatility (~12%)
- 20% drought chance (yield hit + soil stress)

The table shows, per cycle:

- mean total profit (10y)
- p10/p50/p90
- mean soil drift (N,C,S,B)
- count of infeasible runs (violating soil minima)

This is the "trust check" you were concerned about: **infinite-horizon (or cycle) modes are only good if they survive shocks without collapsing soil constraints**.

What "1,2,3" gives you conceptually

1. **Cycle extraction** gives you candidate "eigen-rotations" (steady repeating solutions)
2. **Scenario MC** tells you which eigen-rotations are *robust* vs *fragile*

3. **Finite-horizon DP** tells you whether the best repeating cycle is actually close to the best long plan, or whether the optimal plan is fundamentally “transient” (rehab then shift).

What to do next (no new scope, just tightening)

ChatGPT 5.2

- extract **top 10 cycles**
- compute each cycle’s:
  - average reward
  - **net soil drift per cycle**
  - **probability of infeasibility under shocks**
  - and a “risk-adjusted score” (e.g. mean − 0.5×std, or maximize p10)

That gives you the practical “pick this rotation” recommendation under carbon-farming + onsite-only.

(Also: if you want livestock to actually show up more, we’ll tune just 2 numbers: grazing revenue and compaction penalty by season/wetness — that’s usually the switch.)